Trabajo Práctico Obligatorio Nro 3

Servidores Web

Desarrollar una API Rest para el dominio seleccionado en el TPO 2. La misma, debe cumplir con las siguientes características, aplicándola al dominio de cada grupo:

1. Proveer al menos un endpoint para crear y otro para actualizar recursos. Por ejemplo, para crear una mascota: **POST /api/mascotas**, y para actualizar una mascota **PUT /api/mascotas/:idCasa**.
2. Proveer un endpoint para obtener un único recurso (por ejemplo, obtener un usuario por id). En caso de no encontrarse, el servidor debe responder con un error con código **404**.
3. Proveer un endpoint para obtener varios recursos, donde se permita indicar la cantidad de elementos a devolver. Por ejemplo, **GET /api/mascotas?cantidad=10&from=0** devuelve las primeras 10 mascotas (para obtener las 10 siguientes, se usaría GET /api/mascotas?cantidad=10&from=10).
4. Cada **endpoint** debe validar los datos de entrada. Por ejemplo, si para crear una **mascota** es necesario el nombre de ella, el servidor debe validar que el campo “nombre” sea enviado y sea del tipo esperado (string). De lo contrario, deberá devolver un error con código **400** con un mensaje explicando lo sucedido.
5. El servidor debe poder “servir” los archivos html/css/javascript desarrollados en el TPO2.
6. **Actualizar** los archivos del TPO 2 para poder consumir la API desarrollada en este TP.
    a. Importante: para obtener los datos, la Web debe usar los endpoints paginados como **GET /api/mascotas?cantidad=10** en lugar de traer todo junto!

Consideraciones:

1. Preferentemente, el servidor será desarrollado con Express.js, o directamente en NodeJS.
2. Todos los endpoints deben comenzar con **/api/** seguido del nombre del recurso en plural (por ejemplo, **/api/mascotas**). Esto aplica incluso cuando se obtiene un único elemento: **GET /api/mascotas/:idMascota**.
3. Para la entrega obligatoria no es necesario utilizar una base de datos.

4. No escriban todo el código en un único archivo .js! Utilice alguna estructura de carpetas que permita dividir responsabilidades. Por ejemplo, “*routes*”, “*services*”, “*controllers*”, “*middlewares*”, “*config*”, “*public*”.
5. Devuelvan errores *claros* que indiquen qué falló, respetando la convención de status codes de: https://en.wikipedia.org/wiki/List_of_HTTP_status_codes.

Desafíos:

La siguiente lista de ítems es totalmente opcional por lo que no forma parte de la nota final. Están pensados principalmente para aquellos estudiantes que quieran aprovechar el Laboratorio para investigar y experimentar con nuevas herramientas. Si el tiempo les permite, sugerimos que al menos intenten alguno de los desafíos, ya que puede ser muy útil para la carrera de cada uno! Sino, les queda como guía para proyectos futuros que quieran realizar.

1. Implementar una base de datos, ya sea relacional (por ejemplo, PostgresSQL o MySQL) o no relacional (por ejemplo, MongoDB). El acceso a los datos puede hacerse directamente a la base de datos o mediante un ORM (por ejemplo, Prisma).
2. Implementar registro y login de usuarios utilizando **JWT**. Opcionalmente, agregar **refresh tokens**.
3. Validar los datos utilizando alguna librería como JOI o ZOD.
4. Documentar la API utilizando Swagger o una herramienta similar.
5. Utilizar **Typescript** en vez de Javascript, definiendo clases e interfaces apropiadamente.
6. Implementar **soft deletes** en vez de borrar los registros.
7. Utilizar Docker para el proyecto.
8. Hacer el deploy del proyecto en algún Cloud Provider como Vercel, Netlify, o AWS EC2.